

SafePlus

Experiment 01 阶段性实验报告

Detection-Time Survival Baseline

GRU / LSTM 对照与论文思路验证

实验分支: `exp/01-lstm-baseline`

阶段结论: 已完成 detection-time survival baseline 的数据核对、LSTM 接入、训练验证、GRU 对照、early 指标核对与个体风险曲线分析。

一、实验背景：从论文问题到本阶段任务

Alex Smola 在《Survival Analysis for Fraud Detection》中区分了欺诈发生时间 t_f 与平台检测时间 t_d ，并强调 $t_f \leq t_d$ 。论文的关键并非再做一个分类器，而是把“欺诈发生”和“欺诈被检测”视为两个不同的时间过程： t_f 是潜变量， t_d 是现实中可观察的标签。

论文第 2.1 节首先讨论用于检测时间的生存分析 (Survival Analysis for Detection)：以 t_d 为已知监督信号，通过 detection hazard 描述用户何时更可能被检测。该模型能够刻画 detection dynamics，但不能直接回答 fraud 从何时开始，因此拟合 t_d 并不等于恢复 t_f 。

因此，本阶段先实现 detection-time baseline：利用用户历史行为 $\mathbf{x}$ 建模 $\lambda_d(t)$ 。目的有二：一是验证数据、survival loss 与训练流程是否闭环；二是建立后续联合模型的对照基线。

二、论文概念与当前实验实现的对应关系

论文概念	当前实验中的对应实现	阶段状态
用户行为序列 $\mathbf{x}$	synthetic 数据中的 $\mathbf{x}$ ，形状为 (2000, 24, 8)	已使用
检测时间 t_d	<code>t_detect</code> ；平台检测/响应时间	已作为监督信息使用
欺诈发生时间 t_f	<code>t_onset</code> ；synthetic 中已知但训练阶段不作为监督输入	仅用于评估/对照
检测风险 $\lambda_d(t)$	LSTM-R / SAFE-R 输出的离散时间 detection hazard / cumulative risk	已建模
欺诈风险 $\lambda_f(t)$	Normal $\rightarrow$ Fraud 的发生风险	本阶段未实现
联合模型 $p(t_f, t_d \mathbf{x})$	论文第 2.3 节核心	下一阶段
后验 $p(t_f t_d, \mathbf{x})$	根据已知 t_d 和 $\mathbf{x}$ 反推潜在 t_f	下一阶段

当前 LSTM-R 不是论文最终的 SafePlus 联合模型，而是论文第 2.1 节 detection-time survival 思路的离散时间 baseline。本阶段只回答“利用 t_d 监督能学到什么”，不涉及 t_f 的直接推断。

三、Synthetic 数据：为什么先在“知道答案”的环境中验证

真实欺诈数据中 t_f 往往不可直接观察。论文指出，公开数据通常能够提供检测/封禁时间 t_d ，或某次已观测的欺诈行为时间，但很少同时提供可靠的 t_f 与 t_d 。因此本阶段先用 synthetic 数据做受控验证：生成时保留真实 onset，训练时不把 `t_onset` 作为监督信号，实

验结束后再用它检查模型是否真正提前。

本次数据规模为 $\text{shape}(\mathbf{x}) = (2000, 24, 8)$ ，即 2000 个实体、每个实体 24 个时间步、每步 8 维特征。欺诈样本从 t_{onset} 开始逐渐叠加异常信号，因此 t_f 更接近“行为分布开始偏移”的时间，而不是某个特征瞬间跳高的单点。

```
Anaconda Prompt
(base) F:\research\SafePlus>python scripts\01_inspect_synthetic.py
x shape: (10, 24, 8)
lengths shape: (10,)

10000:
user= 0 | detected=0 | tf=-1 | td=23
user= 3 | detected=0 | tf= 6 | td=23
user= 5 | detected=1 | tf=12 | td=19
user= 9 | detected=1 | tf=12 | td=13

Inspect user 5:
t=11 | feature_mean= 0.696
t=12 | feature_mean= 0.684 <-- tf
t=16 | feature_mean= 1.752
t=18 | feature_mean= 1.465
t=19 | feature_mean= 0.455 <-- td
t=23 | feature_mean= 1.011

(base) F:\research\SafePlus>python -c "...np.load(...); print shapes..."
x: (2000, 24, 8)
detected: (2000,)
tf: (2000,)
td: (2000,)
```

图 1 Synthetic 数据结构与 t_f/t_d 样本检查

四、Detection-Time Survival Baseline 的实现

现有仓库中的 SAFER 使用 GRU 作为时序编码器。本阶段保留原实现，并新增 LSTMSequenceEncoder 与 LSTMSAFER，形成可直接对照的 GRU/LSTM detection-time survival baseline。除编码器外，数据、survival loss、训练器、随机种子和主要超参数保持一致。

LSTM-R 的前向过程可写为

$$\mathbf{x}_{1:t} \xrightarrow{\text{LSTM}} \mathbf{h}_t \longrightarrow h_d(t). \quad (1)$$

其中， $\mathbf{x}_{1:t}$ 是截至当前时刻的行为历史， $\mathbf{h}_t$ 是 LSTM 的内部状态， $h_d(t)$ 表示在此前尚未发生 detection 的条件下，当前时间步发生 detection 的概率。代码采用离散时间形式， $h_d(t)$ 与论文连续时间风险率 $\lambda_d(t)$ 在表达上不同，但建模角色一致。

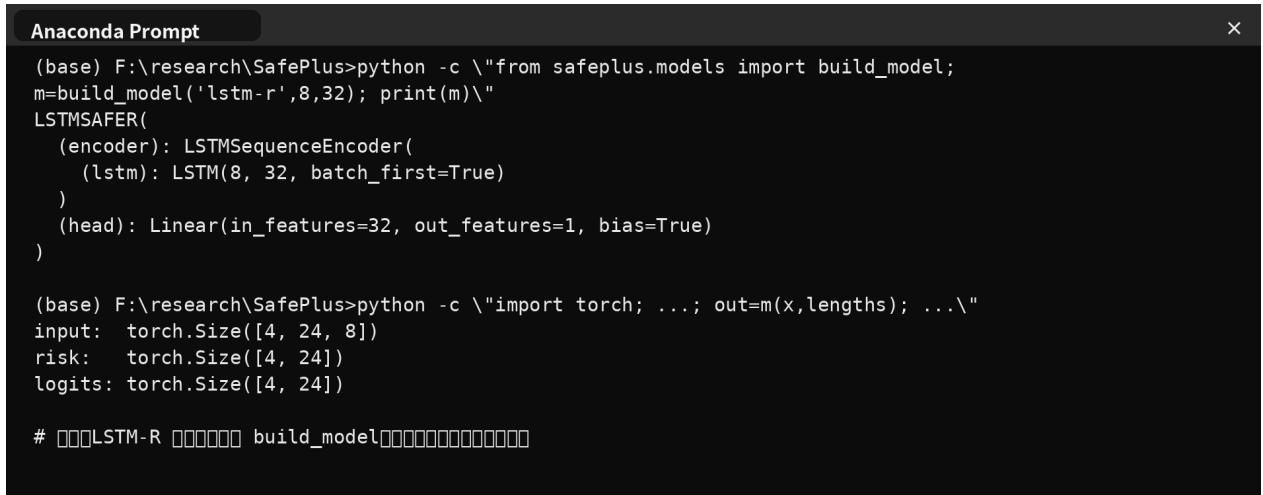
若用户在 d 时刻被检测，离散时间 likelihood 为

$$\Pr(t_d = d) = h_d(d) \prod_{j=1}^{d-1} [1 - h_d(j)]. \quad (2)$$

若观察截至 T 时刻仍未检测，则使用右删失项

$$\Pr(t_d > T) = \prod_{j=1}^T [1 - h_d(j)]. \quad (3)$$

这与论文第 2.1 节“检测前持续生存、检测时刻发生事件”的逻辑一致。上述连乘式采用从 1 开始的时间步编号；若代码从 0 开始计数，相应下限改为 0。



```

Anaconda Prompt
(base) F:\research\SafePlus>python -c "from safeplus.models import build_model;
m=build_model('lstm-r',8,32); print(m)\n"
LSTMSAFER(
  (encoder): LSTMSequenceEncoder(
    (lstm): LSTM(8, 32, batch_first=True)
  )
  (head): Linear(in_features=32, out_features=1, bias=True)
)

(base) F:\research\SafePlus>python -c "import torch; ...; out=m(x,lengths); ...\n"
input:  torch.Size([4, 24, 8])
risk:    torch.Size([4, 24])
logits:  torch.Size([4, 24])

#  LSTM-R  build_model

```

图 2 LSTM-R 模型结构与前向传播验证

五、实验配置与可复现性

为保证对照可复现，本阶段新增 `configs/synthetic/lstm-r.yaml`，并沿用 SAFE-R 的主要训练设置。配置与模型代码分离后，只需切换 `model.name` 即可复现实验，也能减少手工改源码带来的额外变量。

配置项	当前设置
数据	data/processed/synthetic_demo.npz
数据规模	$2000 \times 24 \times 8$
Train / Val / Test	0.7 / 0.1 / 0.2
Seed	0
Hidden dim	32
Batch size	64
Learning rate	0.001
Max epochs / patience	40 / 7
Device	CPU

六、单模型结果：结果符合基线预期，LSTM-R 未引入显式预警训练目标

LSTM-R 在 `seed = 0` 下正常训练并输出 held-out 指标。AUROC ≈ 0.9322 、AUPRC ≈ 0.7147 、Recall ≈ 0.9688 ，说明模型能够较好地地区分最终高风险用户。这一结果符合本阶段对 detection-time baseline 的预期：LSTM-R 的训练目标仍然是解释检测时间 t_d ，而不是直接恢复 t_f 。

但 early 指标明显偏弱：`early_detected_rate = 0.03125`，`mean_lead_time` 为负，F1@1 至 F1@5 均为 0。这并不意味着模型训练失败，而是与其训练目标一致：LSTM-R 没有使用 `t_onset(t_f)` 作为监督，也没有额外加入“在 t_d 之前必须报警”的预警损失或训练机制，因此模型主要学习的是 detection dynamics。

```
Anaconda Prompt
(base) F:\research\SafePlus>type runs\synthetic\lstm-r\seed_0\metrics.json
{
  "accuracy": 0.8875,
  "accuracy@1": 0.76,
  "accuracy@2": 0.76,
  "accuracy@3": 0.76,
  "accuracy@4": 0.76,
  "accuracy@5": 0.76,
  "auprc": 0.7146964703471447,
  "auroc": 0.9321546052631579,
  "early_detected_rate": 0.03125,
  "f1": 0.8051948051948052,
  "f1@1": 0.0, "f1@2": 0.0, "f1@3": 0.0, "f1@4": 0.0, "f1@5": 0.0,
  "mean_lead_time": -6.118279569892473,
  "precision": 0.6888888888888889,
  "recall": 0.96875,
  "threshold": 0.43999999999999995
}
```

图 3 LSTM-R seed = 0 的 held-out 指标

七、GRU / SAFE-R 与 LSTM-R 对照

为判断 early detection 较弱究竟来自编码器能力，还是 detection-only 目标本身，本阶段在相同 synthetic 数据、seed = 0 和主要训练参数下复现 SAFE-R (GRU)，并与 LSTM-R 进行对照。

指标	GRU / SAFE-R	LSTM-R	观察
Accuracy	0.8925	0.8875	接近
AUROC	0.9276	0.9322	LSTM 略高
AUPRC	0.6975	0.7147	LSTM 略高
Precision	0.7023	0.6889	GRU 略高
Recall	0.9583	0.9688	LSTM 略高
F1	0.8106	0.8052	接近
Early detected rate	0.03125	0.03125	相同
Mean lead time	-6.5761	-6.1183	LSTM 稍好，但仍为负
F1@1 至 F1@5	均为 0	均为 0	早期识别均弱

GRU/SAFE-R 的 held-out 指标如下，用于与 LSTM-R 的结果直接对照。

```
(base) F:\research\SafePlus>type runs\synthetic\safe-r\seed_0\metrics.json
{
  "accuracy": 0.8925,
  "accuracy@1": 0.76,
  "accuracy@2": 0.76,
  "accuracy@3": 0.76,
  "accuracy@4": 0.76,
  "accuracy@5": 0.76,
  "auprc": 0.6975195748322444,
  "auroc": 0.9275973135964912,
  "early_detected_rate": 0.03125,
  "f1": 0.8105726872246696,
  "f1@1": 0.0,
  "f1@2": 0.0,
  "f1@3": 0.0,
  "f1@4": 0.0,
  "f1@5": 0.0,
  "mean_lead_time": -6.576086956521739,
  "precision": 0.7022900763358778,
  "recall": 0.9583333333333334,
  "threshold": 0.41999999999999993
}
```

图 4 GRU/SAFE-R seed = 0 的 held-out 指标

结果显示，两者在普通分类指标上接近：LSTM 的 AUROC、AUPRC 和 Recall 略高，但 `early_detected_rate` 完全相同，mean lead time 仍为负。这一结果同样符合预期：GRU/SAFE-R 与 LSTM-R 使用的是同一类 detection-time survival 目标，改变编码器并不会自动改变监督信号，也不会凭空产生针对 t_f 的预警能力。

这一现象与论文对 detection-only survival model 的分析一致：模型可以学习 detection dynamics，却没有显式建模 fraud commission time。当前实验不能据此证明联合模型一定更优，但它说明“更强的序列编码器”和“更合理的时间建模目标”是两件不同的事。

八、代表性样本：风险曲线与 t_f / t_d 的时间错位

为进一步观察时间上的错位，选取测试样本 entity 1067。该用户真实 $t_f = 4$ 、 $t_d = 12$ ，即欺诈从第 4 个时间步开始，但平台到第 12 个时间步才检测到。

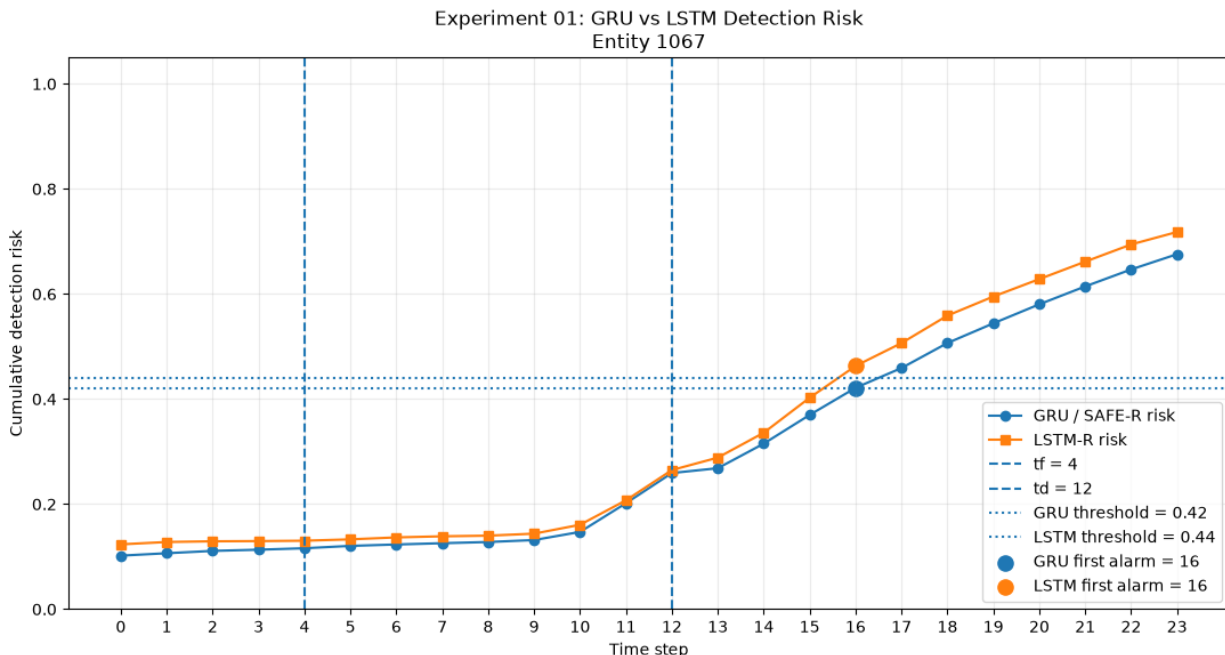


图 5 Entity 1067: GRU / LSTM 累计 detection risk 与 t_f 、 t_d 对齐

图中，GRU 和 LSTM 的累计 detection risk 在 $t_f = 4$ 附近均未明显上升；到 $t_d = 12$ 时，两条曲线仍低于报警阈值。两种模型都在 $t = 16$ 才首次越过阈值，比真实 detection time 晚 4 个时间步。这与整体 mean lead time 为负的结果一致。

entity 1067 只是用于解释晚报警现象的代表性案例，不能替代总体统计。因此，个体曲线用于解释“现象如何发生”，`early_detected_rate` 和 `mean_lead_time` 才用于判断问题是否普遍。

九、与论文的对照思考：为什么这一步是必要的 baseline

本阶段把论文第 2.1 节从公式层面落实到了实验层面： t_d 是数据中可观察的标签，而 $\lambda_d(t)$ 是模型在 t_d 监督下根据 $\mathbf{x}$ 学到的 detection risk function。训练的目标不是预测一个未知 t_d ，而是利用已知 t_d 学习不同历史行为对应的 detection dynamics。

这也解释了当前 baseline 的局限。对于 $t_f = 4$ 、 $t_d = 12$ 的用户，训练目标只要求解释 detection time，并没有直接告诉模型“4 是 onset”。因此，更强的 encoder 与更合理的时间建模目标是两件不同的事。

论文第 2.3 节进一步提出 joint model：用 $\lambda_f(t)$ 描述 Normal $\rightarrow$ Fraud，用 $\lambda_d(t)$ 描述 Fraud $\rightarrow$ Detected，并满足 $t_f \leq t_d$ ；由于 t_f 不可直接观察，还需进一步推断 $p(t_f | t_d, \mathbf{x})$ 。从当前 baseline 的结果看，下一步更值得验证的是时间建模目标本身，而不是继续单纯增

加编码器复杂度。

十、阶段性结论与下一步

1. Detection-time survival baseline 已形成可复现实验闭环：synthetic 数据读取、模型构建、survival loss、训练、评估、预测与可视化均已跑通。
2. LSTM-R 的最终风险识别能力较好，但 `early_detected_rate` 很低、mean lead time 为负。该结果符合本实验设定：LSTM-R 没有显式的预警训练目标，主要围绕 t_d 学习 detection dynamics。
3. GRU 与 LSTM 的 early 指标接近，当前没有证据表明仅提升序列编码器复杂度就能解决 early detection。
4. 当前现象与论文对 detection-only survival model 的局限分析一致，但结果仅来自 single seed 和 synthetic 数据，应视为阶段性观察，而非普遍结论。
5. 下一实验进入 dual-hazard：同时建模 $\lambda_f(t)$ 与 $\lambda_d(t)$ ，进一步实现 latent t_f 的 posterior $p(t_f | t_d, \mathbf{x})$ ，检验是否能把报警时间从 t_d 拉向 t_f 。

十一、可复现记录与 GitHub 阶段封存

当前实验分支为 `exp/01-lstm-baseline`。首轮 LSTM baseline 已推送，commit 为 `e82f193`。本轮新增 GRU 对照、prediction 输出、风险曲线与阶段报告后，可再次 commit/push，作为后续 dual-hazard 实验的固定基线版本。

参考资料

Alex Smola. *Survival Analysis for Fraud Detection*. Technical note, 2022.

Panpan Zheng, Shuhan Yuan, Xintao Wu. *SAFE: A Neural Survival Analysis Model for Fraud Early Detection*. AAAI, 2019.

How Jing, Alexander J. Smola. *Neural Survival Recommender*. WSDM, 2017.